Guía de exposición — Fernando Palencia

Curso: Desarrollo Web · **Proyecto:** Vivent · **Ciclo:** octubre 2026

Documento de apoyo individual. No sustituye al `DOCUMENTO_PROYECTO.md`; lo resume desde tu rol.

---

0. Tu rol en una tabla

Elemento	Lo que te corresponde
Módulos (2)	Traslados · Ajustes de inventario
Proceso integrado (1)	P3 — Traslado entre sucursales con reserva, despacho y recepción
Reportes (3)	Traslados por estado · Ajustes de inventario · Movimientos de inventario
API externa (1)	**open.er-api.com** (tipo de cambio)
Transversales	Datos de prueba (`bootstrap_demo.py`) · Revisión de seguridad
Carpetas de Postman	`05 · Inventario` (Movimientos + **Ajustes**), `08 · Traslados`, `10 · Reportes`, `11 · Integraciones externas` (tipo de cambio), `12 · Usuarios y permisos` (demo de 403)

> Tu proceso es el único que **cruza dos sucursales**: la reserva se hace en el origen y la existencia se mueve al recibir. Es el proceso más vistoso para la defensa porque se ve el stock cambiar en dos lugares.

---

1. Guion de 12 minutos

Min	Qué muestras	Qué dices
0–1	Login como `transfers_destination` → Traslados → **Nuevo traslado**	"La sucursal destino solicita; el origen no puede ser la misma sucursal y no se puede pedir a otra empresa."
1–3	Elegir origen, producto y cantidad → solicitar → estado `REQUESTED`	"Todavía no se movió nada: solo hay una solicitud."
3–5	Login como `admin` → **Aprobar**	"Al aprobar se **reserva** stock: `quantity_reserved` sube, `quantity_available` baja y `on_hand` no cambia. Por eso nadie más puede vender ese stock."
5–6	Login como `transfers_origin` → **Despachar**	"El despacho exige permiso distinto al de aprobación: separación de funciones. El estado pasa a `IN_TRANSIT`."
6–8	Volver como `transfers_destination` → **Recibir** (cantidad completa o parcial)	"Al recibir se descuenta del origen, se libera la reserva y entra en el destino; quedan **dos** movimientos de kardex, uno por sucursal."
8–10	Ajustes → nuevo ajuste tipo `DAMAGE` dirección `OUT` → enviar a aprobación → aprobar como `admin`	"El ajuste no toca el stock hasta que se aprueba; al aprobar genera `DAMAGE_OUT` y queda en auditoría con motivo."
10–11	Reportes: Traslados por estado (filtro `IN_TRANSIT` / `RECEIVED`) → **Exportar CSV** → **Imprimir**	"El reporte consolida por estado y sucursal; el CSV respeta el filtro."
11–12	Postman `11` → tipo de cambio USD→GTQ y moneda inválida	"Integración real con open.er-api.com, con caché de 1 hora y respuesta `available=false` si la moneda no existe."

---

2. Postman: qué ejecutar y en qué orden

#	Request exacto	Resultado esperado	Qué demuestras
1	`01 · Autenticación → Login (guarda tokens)`	`200`	Sesión
2	`08 · Traslados → Sucursales disponibles como origen`	`200`	Aislamiento por empresa
3	`08 · Traslados → Crear traslado`	`200` estado `REQUESTED`	Solicitud
4	`08 · Traslados → Aprobar traslado`	`200` estado `APPROVED`	**Reserva de stock**
5	`08 · Traslados → Despachar traslado`	`200` estado `IN_TRANSIT`	Separación de funciones
6	`08 · Traslados → Recibir traslado`	`200` estado `RECEIVED`	Movimiento entre sucursales
7	`08 · Traslados → Cancelar traslado (ya recibido)`	`400 INVALID_TRANSFER_STATUS`	Máquina de estados
8	`05 · Inventario → Ajustes → Crear ajuste`	`200`	Ajuste en borrador
9	`05 · Inventario → Ajustes → Aprobar ajuste`	`200`	Genera kardex
10	`05 · Inventario → Ajustes → Rechazar ajuste (ya aprobado)`	`400`	Validación de estado
11	`05 · Inventario → Movimientos`	lista paginada	Reporte de movimientos
12	`10 · Reportes → Resumen de traslados`	`200`	Reporte de traslados
13	`10 · Reportes → Resumen de ajustes`	`200`	Reporte de ajustes
14	`11 · Integraciones externas → Tipo de cambio USD→GTQ`	`available: true`, `rate`	API externa
15	`11 · Integraciones externas → Tipo de cambio: moneda inválida (error)`	`available: false`	Manejo de error

Demo de 403 (revisión de seguridad): con el token de `seller` (permiso `sales.create`, sin `users.view`), llama `GET {{baseUrl}}/api/v1/users` → responde `403 PERMISSION_DENIED`. Es la prueba de que el backend no confía en el frontend.

---

3. Código: dónde apuntar

Tema	Archivo	Punto exacto
Solicitar traslado	`backend/.../transfers/service/TransferService.java`	`createTransfer()` valida empresa, sucursales distintas y líneas
Aprobar = reservar	`backend/.../transfers/service/TransferService.java`	`approveTransfer()` → `inventoryService.reserveStock(...)`
Despachar	`backend/.../transfers/service/TransferService.java`	`dispatchTransfer()` (permiso `transfers.dispatch`)
Recibir	`backend/.../transfers/service/TransferService.java`	`receiveTransfer()` → descuenta origen, libera reserva, suma destino
Rechazar / cancelar	`backend/.../transfers/service/TransferService.java`	`rejectTransfer()`, `cancelTransfer()` con motivo obligatorio
Reserva de stock	`backend/.../inventory/service/InventoryService.java`	`reserveStock()` y `releaseReservedStock()`
Crear ajuste	`backend/.../adjustments/service/InventoryAdjustmentService.java`	`createAdjustment()` (estado inicial `PENDING_APPROVAL` o `DRAFT`)

Tema	Archivo	Punto exacto
Aprobar / rechazar	`backend/.../adjustments/service/InventoryAdjustmentService.java`	`approveAdjustment()` aplica `stockIn` / `stockOut`; `rejectAdjustment()` no toca stock
Tipo de movimiento	`backend/.../adjustments/service/InventoryAdjustmentService.java`	`movementTypeFor()` (DAMAGE_OUT, LOSS_OUT, EXPIRED_OUT…)
Reportes que expones	`backend/.../reports/service/ReportService.java`	`transfersSummary()`, `adjustmentsSummary()`
Movimientos	`backend/.../inventory/service/InventoryService.java`	`listMovements()` con filtros
Seguridad (revisión)	`backend/.../security/SecurityConfiguration.java` · `JwtAuthenticationFilter.java` · `RestAccessDeniedHandler.java`	Cadena de filtros, `401` vs `403`, permisos por endpoint
Datos de prueba	`database/bootstrap_demo.py`	Crea compras, ventas, traslado, ajuste y cierre por API real
Frontend traslados	`frontend/src/features/transfers/`	`transfer-form-screen.tsx`, `transfer-detail-screen.tsx`, `lib/transfer-permissions.ts`
Frontend ajustes	`frontend/src/features/adjustments/`	`adjustment-form-screen.tsx`, `adjustment-detail-screen.tsx`

---

4. Tus tres reportes

Reporte	Ruta	Endpoint que consume	Filtros que debes mover
Traslados por estado	`/reports/transfers`	`GET /api/v1/reports/transfers-summary`	Estado, sucursal origen, sucursal destino, fechas
Ajustes de inventario	`/reports/adjustments`	`GET /api/v1/reports/adjustments-summary`	Estado, sucursal, fechas
Movimientos de inventario	`/reports/movements`	`GET /api/v1/inventory/movements`	Tipo de movimiento, sucursal, producto, fechas

Guion de 60 segundos: filtra por estado `RECEIVED` (o por tipo `TRANSFER_IN`) → muestra que bajan las filas → **Exportar CSV** → **Imprimir** y enseña la vista previa sin menú.

> Dato para lucirte: tras el traslado de la demo hay **movimientos en las dos sucursales** (`TRANSFER_OUT` y `TRANSFER_IN`). Filtra por producto y se ven ambos.

---

5. Tu API externa: tipo de cambio

Dato	Valor
Endpoint	`GET /api/v1/integrations/exchange-rate?base=USD&target=GTQ`
Código	`backend/.../integrations/exchange/controller/ExchangeRateController.java` + `service/ExchangeRateService.java`
Respuesta de éxito	`{ baseCurrency: "USD", targetCurrency: "GTQ", available: true, rate: 7.64…, lastUpdatedUtc, source: "OPEN_ER_API" }`
Respuesta de error	`{ available: false, message }` (moneda inexistente o servicio caído)
Protecciones	Caché 1 hora · timeout 4 s · requiere `settings.view`

Qué decir: "El sistema es multimoneda por compañía; el tipo de cambio se consulta a open.er-api.com y se cachea una hora para no golpear el servicio en cada pantalla. Si la moneda no existe, la API responde `available=false` y el sistema sigue funcionando."

Demo: `USD → GTQ` (éxito) y `USD → XXX` (`available: false`).

---

6. Rúbrica: cómo la cumples tú

Criterio de la rúbrica	Cómo se cumple	Evidencia que muestras
2 módulos funcionales	Traslados (solicitar → aprobar → despachar → recibir → cancelar) y Ajustes (crear → aprobar/rechazar/cancelar)	Recorrido en vivo + `08` y `05` de Postman
1 proceso integrado	P3: traslado con reserva, despacho y recepción en dos sucursales	Los cuatro estados en pantalla + kardex de ambas
3 reportes con filtros	Traslados, Ajustes, Movimientos	Un filtro aplicado en cada uno
Impresión / exportación	Botones de `report-actions.tsx`	CSV descargado + vista previa
1 API externa	open.er-api.com con éxito y error	Requests de la carpeta `11`
≥5 registros por catálogo	Eres el responsable de los datos de prueba: seed `S20261026000001` (8 unidades, 5 categorías, 5+ proveedores/clientes, 10+ presentaciones) + `bootstrap_demo.py`	Conteo en los listados
JWT access + refresh	Tu parte: **seguridad** — rotación/revocación y `403` real	Login `seller` + `GET /users` → 403
CRUD en Postman	Traslados, ajustes y movimientos	Run de las carpetas
Revisión de seguridad	`SecurityConfiguration`, `JwtAuthenticationFilter`, `RestAccessDeniedHandler`	Explicación del 401 vs 403
Commits propios	Pendiente: debes subir tu propia contribución	`git log --author="Fernando"`

---

7. Preguntas probables

1. **¿Por qué reservar si todavía no salió la mercadería?** Para que no se venda dos veces: el origen compromete el stock desde la aprobación, aunque `on_hand` recién baje al recibir.
2. **¿Qué pasa si se pierde mercadería en el camino?** Se recibe parcialmente; el faltante queda registrado en el detalle (`quantity_received_base < quantity_approved_base`) y se documenta con un ajuste.
3. **¿Se puede recibir más de lo aprobado?** No: el servicio valida `recibido ≤ aprobado` y responde `400`.
4. **¿Se puede trasladar entre empresas distintas?** No: las sucursales se cargan con `company_id` del token; una sucursal ajena no existe para la consulta.
5. **¿Por qué aprobar y despachar son permisos distintos?** Separación de funciones: quien autoriza no es quien mueve físicamente. En la demo lo hacen `admin` y `transfers_origin`.
6. **¿El ajuste modifica el stock al crearse?** No, solo al aprobarse; por eso existe el estado `PENDING_APPROVAL` y el permiso `inventory.adjust_approve`.
7. **¿El kardex se puede editar?** Nunca: es un libro de movimientos; las correcciones se hacen con un movimiento nuevo.
8. **¿Cómo se audita?** Cada transición llama a `AuditLogService.log()` con usuario, entidad, valor anterior y nuevo.

---

8. Checklist 10 minutos antes

- [ ] `docker ps` con los tres contenedores arriba.
- [ ] `python database/bootstrap_demo.py` corrido (deja el traslado `ACAD-TRF-001` y el ajuste `ACAD-ADJ-001`).
- [ ] Probaste los tres usuarios: `transfers_destination`, `transfers_origin`, `admin` (todos `ChangeMe123!`).
- [ ] Tienes a mano el token de `seller` para la demo de 403 (o el request de login con otro usuario).
- [ ] `documento-proyecto.pdf` abierto en §5.5–5.6 y §7.4 (flujo de traslado).
- [ ] Pestaña con `/reports/transfers` ya cargada.